

Ultimátní PL/SQL Referenční Příručka

Komplexní přehled syntaxe, šablon a řešení chyb

31. března 2026

Obsah

1	Systémový katalog a obecná pravidla	2
2	SQL Specifika a Analýza dotazů	2
3	Dynamické SQL a Proměnné	2
4	Datové modelování (Oracle Academy)	3
5	Výjimky (Exceptions)	4
6	Úplné Šablony PL/SQL Objektů	5
6.1	1. PROCEDURE (Procedura)	5
6.2	2. FUNCTION (Funkce)	6
6.3	3. TRIGGER (Spouštěč)	7
6.4	4. FOR LOOP (Implicitní kurzor)	7
6.5	5. DYNAMICKÉ SQL (EXECUTE IMMEDIATE)	8

1 Systémový katalog a obecná pravidla

- **Katalog (USER_TABLES, USER_TAB_COLUMNS):** Názvy jsou v databázi zásadně **VELKÝMI PÍSMENY**. Při dynamickém hledání vždy používej UPPER(p_name).
- **Apostrofy vs. Uvozovky:**
 - ' ' (Jednoduché): Pro textové řetězce (např. 'REPORT').
 - "" (Dvojitě): Pro vynucení názvu objektu (v PL/SQL kódu se jim vyhýbej).
- **Escapování apostrofů v Dynamickém SQL:** Pokud potřebuješ do řetězce vložit apostrof, musíš ho zdvojit: `v_sql := 'SELECT * FROM tab WHERE jmeno = ''Karel''';`
- **Správa místa:** Pokud dojde kvóta (ORA-01652), použij `PURGE RECYCLEBIN;` nebo maž tabulky přes `DROP TABLE nazev PURGE;`.
- **Operátor LIKE:** Vyžaduje zástupný znak % (např. `LIKE 'J_REPORT%'`).

2 SQL Specifika a Analýza dotazů

- **Pořadí zpracování (Problém Alias):** Klauzule HAVING se vyhodnocuje **před** klauzulí SELECT. Proto v HAVING nelze použít alias (např. `article_count`). Musíš tam zopakovat celou agregační funkci.
- **CTAS (Create Table As Select):** Každý vypočítaný/agregační sloupec **musí** mít alias (`AS jmeno`), jinak vytvoření tabulky selže na chybě ORA-00998.
- **Filtrování u LEFT JOIN:** Podmínka pro levou tabulku ve WHERE změní spojení na INNER JOIN. Pro zachování prázdných řádků musí být podmínka explicitně v klauzuli ON.
- **Agregace a SYSTIMESTAMP:** Funkce jako SYSTIMESTAMP nebo SYSDATE se nedávají do GROUP BY, jinak rozbijí seskupování.
- **SQL%ROWCOUNT:** Systémový atribut vracející počet řádků ovlivněných posledním UPDATE/DELETE/INSERT. Kontroluj hned po příkazu přes `IF SQL%ROWCOUNT = 0 THEN...`

3 Dynamické SQL a Proměnné

- **Statické SQL:** Uvnitř BEGIN...END musí mít každý databázový SELECT klauzuli INTO.
- **Vázané proměnné (USING):** Lze použít **pouze** v DML příkazech pro předání hodnot. Chrání před SQL Injection a zvyšují výkon.
- **Zákaz v DDL:** V příkazech CREATE, DROP nebo ALTER nelze použít USING. Jména tabulek a sloupců se musí lepit přímo do řetězce přes zřetězení ||.
- **Deklarace typu %TYPE / %ROWTYPE:**
 - `v_jmeno z_author.name%TYPE;` (Dynamicky okopíruje datový typ z tabulky).
 - `v_radek z_article%ROWTYPE;` (Vytvoří krabici se strukturou celého řádku tabulky).

4 Datové modelování (Oracle Academy)

- **Vztah 1:1 (Optional to Mandatory):** Cizí klíč (FK) patří logicky **vždy na povinnou stranu** (Tabulka B).
- **Složený PK:** Pokud tvoříš vazební tabulku M:N: `ALTER TABLE x ADD CONSTRAINT pk_name PRIMARY KEY (col1, col2);`.

5 Výjimky (Exceptions)

Jméno výjimky	Číslo chyby	Popis
NO_DATA_FOUND	ORA-01403	SELECT INTO nevrátil nic. (COUNT ji nikdy nevyvolá!).
TOO_MANY_ROWS	ORA-01422	SELECT INTO vrátil více řádků. Lze řešit kurzorem.
DUP_VAL_ON_INDEX	ORA-00001	Porušení unikátního klíče. Pokus o zápis duplicity.
VALUE_ERROR	ORA-06502	Chyba velikosti/typu (např. příliš dlouhý text do proměnné).

Šablona: Uživatelské výjimky (Vlastní logika)

```
1 DECLARE
2     e_moje_chyba EXCEPTION; -- 1. Deklarace
3 BEGIN
4     IF :NEW.hodnota < 0 THEN
5         RAISE e_moje_chyba; -- 2. Vyvolani vyjimky
6     END IF;
7 EXCEPTION
8     WHEN e_moje_chyba THEN
9         -- 3. Zachyceni a zpracovani
10        dbms_output.put_line('Chyba logiky, nesmi byt zaporne!');
11    WHEN OTHERS THEN
12        -- Vždy zachytej i necekane chyby
13        dbms_output.put_line(SQLERRM);
14 END;
```

Šablona: Rychlá chyba (raise_application_error)

Tento příkaz rovnou zastaví provádění a vyhodí chybu na klienta. Rozsah čísel musí být povinně mezi -20000 až -20999. V triggerech automaticky způsobí ROLLBACK.

```
1 BEGIN
2     IF p_param IS NULL THEN
3         raise_application_error(-20001, 'Parametr nesmi byt NULL!');
4     END IF;
5 END;
```

6 Úplné Šablony PL/SQL Objektů

6.1 1. PROCEDURE (Procedura)

Procedury nevyužívají slovo DECLARE. Vracení hodnot se děje manipulací dat v tabulkách (DML/DDL), nebo přes OUT parametry (bývá vzácné). **Syntaxe:**

```
1 CREATE [OR REPLACE] PROCEDURE jmeno_procedure
2   [ (parametr1 [IN|OUT|IN OUT] datovy_typ, ...) ]
3 IS | AS
4   -- Zde se deklarují proměnné
5 BEGIN
6   -- Vykonný kód
7 [EXCEPTION
8   -- Ošetření chyb]
9 END [jmeno_procedure];
```

Praktický příklad (Smazání + DDL):

```
1 CREATE OR REPLACE PROCEDURE P_priklad(p_id INT, p_mode VARCHAR2) IS
2   v_sql VARCHAR2(2000);
3   v_exists INT;
4 BEGIN
5   -- Zjistění z katalogu
6   SELECT count(*) INTO v_exists FROM user_tables WHERE table_name = 'TEST';
7
8   IF p_mode = 'REPLACE' AND v_exists > 0 THEN
9     EXECUTE IMMEDIATE 'DROP TABLE TEST';
10  END IF;
11
12  -- Pozor: Zadne USING pro DDL, lepi se to natvrdo pres zretezeni
13  v_sql := 'CREATE TABLE TEST AS SELECT * FROM z_article WHERE aid = ' ||
14  p_id;
15  EXECUTE IMMEDIATE v_sql;
16 END P_priklad;
```

6.2 2. FUNCTION (Funkce)

Funkce **musí** obsahovat definici návratového typu a **musí** v každé větvi ukončit svůj běh příkazem RETURN. Záchranný RETURN musí být i v bloku EXCEPTION. **Syntaxe:**

```
1 CREATE [OR REPLACE] FUNCTION jmeno_funkce
2   [ (parametr1 [IN] datovy_typ, ...) ]
3   RETURN navratovy_datovy_typ
4 IS | AS
5   -- Zde se deklarují proměnné
6 BEGIN
7   -- Vykonný kód
8   RETURN hodnota;
9 [EXCEPTION
10   -- Ošetření chyb
11   RETURN hodnota_pro_chybu;]
12 END [jmeno_funkce];
```

Praktický příklad:

```
1 CREATE OR REPLACE FUNCTION F_pocet(p_year INT) RETURN INT IS
2   v_res INT;
3 BEGIN
4   SELECT count(*) INTO v_res FROM z_article WHERE year = p_year;
5   RETURN v_res;
6 EXCEPTION
7   WHEN OTHERS THEN
8     -- Navrat defaultní hodnoty při havarii
9     RETURN -1;
10 END F_pocet;
```

6.3 3. TRIGGER (Spouštěč)

Spouštěč se váže na tabulku a akci (DML). **Zákaz:** V triggeru (nad řádkem) nesmíš používat COMMIT ani ROLLBACK. Pro zrušení akce se volá `raise_application_error`. Modifikace hodnot (:NEW) je povolena **pouze** v triggeru typu BEFORE. **Syntaxe:**

```
1 CREATE [OR REPLACE] TRIGGER jmeno_triggeru
2   {BEFORE | AFTER | INSTEAD OF}
3   {INSERT [OR] UPDATE [OR] DELETE} [OF sloupce]
4   ON jmeno_tabulky
5   [FOR EACH ROW] -- Nutne pro pristup k :OLD a :NEW
6 [DECLARE
7   -- Promenne]
8 BEGIN
9   -- Logika triggeru
10 END [jmeno_triggeru];
```

Praktický příklad (Kombinovaný s predikáty):

```
1 CREATE OR REPLACE TRIGGER T_hlidac
2   BEFORE INSERT OR UPDATE OR DELETE ON z_article_author FOR EACH ROW
3 BEGIN
4   IF INSERTING OR UPDATING THEN
5     IF :NEW.rid IS NULL THEN
6       -- Zablokuje DML a vyhodi error klientovi
7       raise_application_error(-20005, 'RID chybi');
8     END IF;
9     -- Modifikace vkladanych dat pred zapise (jen v BEFORE)
10    :NEW.last_change := SYSTIMESTAMP;
11  END IF;
12
13  IF DELETING THEN
14    -- V DELETING mame pristup pouze k :OLD promennym
15    dbms_output.put_line('Mazu autora: ' || :OLD.rid);
16  END IF;
17 END T_hlidac;
```

6.4 4. FOR LOOP (Implicitní kurzor)

Nejčistší a nejbezpečnější způsob, jak projít více záznamů. Odpadá nutnost manuálního OPEN, FETCH a CLOSE. Datový typ iterátoru (záznamu) databáze vyřeší automaticky. **Syntaxe:**

```
1 BEGIN
2   FOR promenna_zaznam IN ( SELECT_dotaz )
3   LOOP
4     -- Logika provadena pro kazdy radek (pristup pres: promenna_zaznam.
5     sloupec)
6     END LOOP;
7 END;
```

Praktický příklad (Iterace a Dynamické DDL):

```
1 BEGIN
2   FOR r IN (SELECT DISTINCT year FROM z_article ORDER BY year) LOOP
3     dbms_output.put_line('Zpracovavam rok: ' || r.year);
4     -- Dynamicke SQL uvnitr cyklu na zaklade dat z kurzoru
5     EXECUTE IMMEDIATE 'CREATE TABLE R_' || r.year || ' AS SELECT * FROM
6     z_article WHERE 1=0';
7     END LOOP;
8 END;
```

6.5 5. DYNAMICKÉ SQL (EXECUTE IMMEDIATE)

Slouží k vykonávání SQL příkazů, které databáze nemůže zkompilovat předem, protože se jejich přesná podoba (nebo typ příkazu) sestavuje jako textový řetězec až za běhu programu.

Kdy **PŘESNĚ** použít Dynamické SQL (Zlatá pravidla):

- **1. Veškeré DDL operace:** Uvnitř bloku BEGIN...END nelze staticky napsat příkazy jako CREATE, DROP, ALTER nebo TRUNCATE. Vždy se musí obalit do EXECUTE IMMEDIATE.
- **2. Názvy tabulek/sloupců z proměnných:** Pokud jméno tabulky nebo sloupce neznáš natvrdo, ale dostáváš ho zvenčí v proměnné (např. p_table_name), statické SQL nepůjde zkompilovat. Dotaz musíš slepit jako text.
- **3. Měnící se struktura dotazu:** Například když se klauzule WHERE skládá dynamicky podle toho, jaké parametry uživatel zadal.

Kdy ho **NEPOUŽÍVAT** (Kdy stačí běžné Statické SQL):

- Pokud znáš jméno tabulky i sloupce a mění se **pouze hodnoty** (např. hledáš konkrétní rok předaný v parametru). V takovém případě piš klasický SELECT ... INTO ... WHERE rok = p_rok; – databáze si s hodnotami poradí sama.

Syntaxe:

```
1 EXECUTE IMMEDIATE stringovy_prikaz_sql
2   [INTO promenna1, promenna2, ...]
3   [USING hodnota1_pro_bind, hodnota2_pro_bind, ...];
```

Praktický příklad:

```
1 DECLARE
2   v_sql VARCHAR2(2000);
3   v_pocet INT;
4   v_nazev_tabulky VARCHAR2(50) := 'Z_ARTICLE'; -- Dynamicky zjisteny nazev
5 BEGIN
6   -- Jmena tabulek a sloupcu nutno spojit retezcem (||) protoze
7   -- staticke SQL by promennou na miste tabulky nepochopilo.
8   v_sql := 'SELECT count(*) FROM ' || v_nazev_tabulky || ' WHERE year = :rok
9   ';
10
11   -- Samotnou hodnotu pro filtr predame bezpecne pres bind promennou
12   EXECUTE IMMEDIATE v_sql INTO v_pocet USING 2026;
13   dbms_output.put_line('Pocet clanku: ' || v_pocet);
14 END;
```